

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №6
по курсу «Проектирование и реализация баз данных»

Выполнил:

Гашимов И.Ф.

К3239

Проверила:

Говорова М.М.

Санкт-Петербург

2026 г.

Цель: овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

2.1.1 Создание БД и заполнение unicorns

```
learn> db.unicorns.insertMany([
  {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63},
  {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
  {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
  {name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
  {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80},
  {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
  {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
  {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
  {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
  {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
  {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}
])

let document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
db.unicorns.insertOne(document)

db.unicorns.find()

{
  _id: ObjectId('6a101fa48850ada9efabc114'),
  name: 'Horny',
  loves: [ 'carrot', 'papaya' ],
  weight: 600,
  gender: 'm',
  vampires: 63
},
{
  _id: ObjectId('6a101fa48850ada9efabc115'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
},
{
  _id: ObjectId('6a101fa48850ada9efabc116'),
  name: 'Unicrom',
  loves: [ 'energon', 'redbull' ],
  weight: 984,
  gender: 'm',
  vampires: 182
},
{
  _id: ObjectId('6a101fa48850ada9efabc117'),
  name: 'Rooooooodles',
  loves: [ 'apple' ],
  weight: 575,
  gender: 'm',
  vampires: 99
},
{
  _id: ObjectId('6a101fa48850ada9efabc118'),
  name: 'Solnara',
  loves: [ 'apple', 'carrot', 'chocolate' ],
  weight: 550,
  gender: 'f',
  vampires: 80
},
{
  _id: ObjectId('6a101fa48850ada9efabc119'),
  name: 'Ayna',
  loves: [ 'strawberry', 'lemon' ],
  weight: 733,
  gender: 'f',
  vampires: 40
},
{
  _id: ObjectId('6a101fa48850ada9efabc11a'),
  name: 'Kenny',
  loves: [ 'grape', 'lemon' ],
  weight: 690,
  gender: 'm',
  vampires: 39
},
{
  _id: ObjectId('6a101fa48850ada9efabc11b'),
  name: 'Raleigh',
  loves: [ 'apple', 'sugar' ],
  weight: 421,
  gender: 'm',
  vampires: 2
},
{
  _id: ObjectId('6a101fa48850ada9efabc11c'),
  name: 'Leia',
  loves: [ 'apple', 'watermelon' ],
  weight: 601,
  gender: 'f',
  vampires: 33
},
{
  _id: ObjectId('6a101fa48850ada9efabc11d'),
  name: 'Pilot',
  loves: [ 'apple', 'watermelon' ],
  weight: 650,
  gender: 'm',
  vampires: 54
},
{
  _id: ObjectId('6a101fa48850ada9efabc11e'),
  name: 'Nimue',
  loves: [ 'grape', 'carrot' ],
  weight: 540,
  gender: 'f',
  vampires: 0
}
```

Создана база данных `learn`. В коллекцию `unicorn` добавлены документы о единорогах. Один документ (`Dunx`) добавлен вторым способом: сначала создан объект `document`, затем он вставлен в коллекцию. Проверка через `find()` показала наличие добавленных документов.

2.2.1 Выборка самцов, самок, carrot

Запрос для вывода списка самцов: `db.unicorns.find({gender: 'm'}).sort({name: 1})`

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a101fa48850ada9efabc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a101fa48850ada9efabc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  }
]
```

Запрос для вывода самок с ограничением первые 3 особи: `db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)`

```
learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a101fa48850ada9efabc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a101fa48850ada9efabc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a101fa48850ada9efabc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
```

Найдем самок, которые любят морковь:

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'})
[
  {
    _id: ObjectId('6a101fa48850ada9efabc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a101fa48850ada9efabc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a101fa48850ada9efabc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Ограничим список с помощью findOne и limit(1):

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a101fa48850ada9efabc115'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a101fa48850ada9efabc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

2.2.2 Самцы без предпочтений и пола

```
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0}).sort({name: 1})
[
  {
    _id: ObjectId('6a101fa48850ada9efabc11f'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  },
  {
    _id: ObjectId('6a101fa48850ada9efabc114'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a101fa48850ada9efabc11a'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a101fa48850ada9efabc11d'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
]
```

Запрос выводит самцов, но из результата исключены поля loves и gender

2.2.3 Обратный порядок добавления

```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a101fa48850ada9efabc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a101fa48850ada9efabc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a101fa48850ada9efabc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a101fa48850ada9efabc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
]
```

Список единорогов выведен в обратном порядке добавления с помощью сортировки по \$natural: -1.

2.2.4 Первое предпочтение без _id

```
learn> db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
```

Для каждого единорога выведено имя и только первое значение массива loves. Поле _id исключено из результата. Для ограничения использован оператор \$slice.

2.3.1 Самки весом от 500 до 700 кг

```
learn> db.unicorns.find(
|   {gender: 'f', weight: {$gte: 500, $lte: 700}},
|   {_id: 0}
| )
[
|   {
|     name: 'Solnara',
|     loves: [ 'apple', 'carrot', 'chocolate' ],
|     weight: 550,
|     gender: 'f',
|     vampires: 80
|   },
|   {
|     name: 'Leia',
|     loves: [ 'apple', 'watermelon' ],
|     weight: 601,
|     gender: 'f',
|     vampires: 33
|   },
|   {
|     name: 'Nimue',
|     loves: [ 'grape', 'carrot' ],
|     weight: 540,
|     gender: 'f'
|   }
| ]
```

Выведены самки единорогов с весом от 500 до 700 кг. Использованы операторы \$gte, \$lte и поле _id исключено.

2.3.2 Самцы от 500 кг, любят grape и lemon

```
learn> db.unicorns.find(
|   {gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}},
|   {_id: 0}
| )
[
|   {
|     name: 'Kenny',
|     loves: [ 'grape', 'lemon' ],
|     weight: 690,
|     gender: 'm',
|     vampires: 39
|   }
| ]
```

Найдены самцы весом от 500 кг, у которых в массиве loves одновременно присутствуют grape и lemon. Для проверки одновременного наличия элементов массива использован оператор \$all.

2.3.3 Единороги без ключа vampires

```
learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a101fa48850ada9efabc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Найдены документы, в которых отсутствует поле vampires. Для проверки использовали \$exists: false.

2.3.4 Имена самцов и первое предпочтение

```
learn> db.unicorns.find(
|   {gender: 'm'},
|   {_id: 0, name: 1, loves: {$slice: 1}}
| ).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
```

Выведен упорядоченный по имени список самцов. Для каждого самца показано имя и первое предпочтение из массива loves.

3.1.1 Коллекция towns и вложенный объект mayor

```
learn> db.towns.find({"mayor.party": "I"}, {_id: 0, name: 1, mayor: 1})
|
| [
|   { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
| ]
learn> db.towns.find({"mayor.party": {$exists: false}}, {_id: 0, name: 1, mayor: 1})
| [ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn> |
```

Создана коллекция towns, содержащая документы с вложенным объектом mayor. Для обращения к вложенному полю использован оператор точка: mayor.party. Первый запрос вывел город с независимым мэром, второй город, где поле mayor.party отсутствует.

3.1.2 Функция, курсор, forEach

```
learn> function maleUnicornsQuery() {
|   return {gender: 'm'};
| }
|
| let cursor = db.unicorns.find(maleUnicornsQuery()).sort({name: 1}).limit(2);
|
| cursor.forEach(function(obj) {
|   printjson(obj);
| });
| {
|   _id: ObjectId('6a101fa48850ada9efabc11f'),
|   name: 'Dunx',
|   loves: [
|     'grape',
|     'watermelon'
|   ],
|   weight: 704,
|   gender: 'm',
|   vampires: 165
| }
| {
|   _id: ObjectId('6a101fa48850ada9efabc114'),
|   name: 'Horny',
|   loves: [
|     'carrot',
|     'papaya'
|   ],
|   weight: 600,
|   gender: 'm',
|   vampires: 63
| }
| }
```

Создана JavaScript-функция, возвращающая условие выборки самцов. На основе запроса создан курсор, результаты отсортированы по имени и ограничены двумя документами. Вывод выполнен через forEach.

3.2.1 Количество самок весом 500-600 кг

```
learn> db.unicorns.countDocuments({gender: 'f', weight: {$gte: 500, $lte: 600}})
2
```

Подсчитано количество самок весом от 500 до 600 кг.

3.2.2 Список предпочтений

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

Получен список уникальных предпочтений из массива loves. Для этого использовали метод distinct().

3.2.3 Количество особей обоих полов

```
learn> db.unicorns.aggregate([
|   {$group: {_id: "$gender", count: {$sum: 1}}}
| ])
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
```

С помощью агрегированного запроса выполнена группировка по полю gender. Оператор \$sum подсчитал количество документов в каждой группе.

3.3.1 Добавление Barny

```
learn> db.unicorns.insertOne({name: 'Barny', loves: ['grape'], weight: 340, gender: 'm'})
| db.unicorns.find({name: 'Barny'})
[
  {
    _id: ObjectId('6a1027518850ada9efabc126'),
    name: 'Barny',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
```

В коллекцию unicorns добавлен новый документ о единороге Barny. Проверка через find() подтвердила добавление документа.

3.3.2 Изменение Айна

```
learn> db.unicorns.updateOne(
|   {name: 'Ayna'},
|   {$set: {weight: 800, vampires: 51}}
| )
|
| db.unicorns.find({name: 'Ayna'})
|
| [
|   {
|     _id: ObjectId('6a101fa48850ada9efabc119'),
|     name: 'Ayna',
|     loves: [ 'strawberry', 'lemon' ],
|     weight: 800,
|     gender: 'f',
|     vampires: 51
|   }
| ]
```

Для документа Айна изменены значения полей weight и vampires. Использован оператор \$set, который обновляет указанные поля без замены всего документа.

3.3.3 Изменение Raleigh

```
learn> db.unicorns.updateOne(
|   {name: 'Raleigh'},
|   {$set: {loves: ['redbull']}}
| )
|
| db.unicorns.find({name: 'Raleigh'})
|
| [
|   {
|     _id: ObjectId('6a101fa48850ada9efabc11b'),
|     name: 'Raleigh',
|     loves: [ 'redbull' ],
|     weight: 421,
|     gender: 'm',
|     vampires: 2
|   }
| ]
```

Для единорога Raleigh обновлено поле loves: теперь в нём указано значение redbull. Использован оператор \$set.

3.3.4 Всем самцам +5 vampires

```
learn> db.unicorns.updateMany(
|   {gender: 'm'},
|   {$inc: {vampires: 5}}
| )
|
| db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, vampires: 1})
|
| [
|   { name: 'Horny', vampires: 68 },
|   { name: 'Unicrom', vampires: 187 },
|   { name: 'Rooooooodles', vampires: 104 },
|   { name: 'Kenny', vampires: 44 },
|   { name: 'Raleigh', vampires: 7 },
|   { name: 'Pilot', vampires: 59 },
|   { name: 'Dunx', vampires: 170 },
|   { name: 'Barney', vampires: 5 }
| ]
```

Всем самцам увеличено значение поля `vampires` на 5. Использован оператор `$inc`, который изменяет числовое поле на заданную величину.

3.3.5 Portland: мэр беспартийный

```
learn> db.towns.updateOne(
|   {name: 'Portland'},
|   {$unset: {"mayor.party": ""}}
| )
|
| db.towns.find({name: 'Portland'}, {_id: 0})
| [
|   {
|     name: 'Portland',
|     population: 528000,
|     last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
|     famous_for: [ 'beer', 'food' ],
|     mayor: { name: 'Sam Adams' }
|   }
| ]
```

Из вложенного объекта `mayor` у города `Portland` удалено поле `party`. Использован оператор `$unset` и обращение к вложенному полю через точку.

3.3.6 Pilot любит chocolate

```
learn> db.unicorns.updateOne(
|   {name: 'Pilot'},
|   {$push: {loves: 'chocolate'}}
| )
|
| db.unicorns.find({name: 'Pilot'}, {_id: 0, name: 1, loves: 1})
| [ { name: 'Pilot', loves: [ 'apple', 'watermelon', 'chocolate' ] } ]
```

В массив `loves` документа `Pilot` добавлено новое значение `chocolate`. Использован оператор `$push`.

3.3.7 Aurora любить sugar и lemon

```
learn> db.unicorns.updateOne(
|   {name: 'Aurora'},
|   {$addToSet: {loves: {$each: ['sugar', 'lemon']}}}
| )
|
| db.unicorns.find({name: 'Aurora'}, {_id: 0, name: 1, loves: 1})
| [ { name: 'Aurora', loves: [ 'carrot', 'grape', 'sugar', 'lemon' ] } ]
```

В массив `loves` документа `Aurora` добавлены значения `sugar` и `lemon`. Использован `$addToSet` с `$each`, чтобы добавить несколько значений без дублирования уже существующих элементов.

3.4.1 Удаление данных из towns

```
learn> db.towns.deleteMany({"mayor.party": {$exists: false}})
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a102a4f8850ada9efabc128'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a102a4f8850ada9efabc129'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```

```
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
learn> db.towns.find()
learn> |
```

```
learn> show collections
towns
unicorns
```

Удалены документы, в которых отсутствует поле mayor.party. Затем коллекция towns очищена полностью командой deleteMany({}). Проверка через find() показала отсутствие документов в коллекции.

4.1.1 Ссылки на зоны обитания

```
learn> db.habitats.find()
[
  {
    _id: 'forest',
    name: 'Лесная зона',
    description: 'Территория с лесной растительностью'
  },
  {
    _id: 'mountain',
    name: 'Горная зона',
    description: 'Высокогорная область'
  },
  {
    _id: 'lake',
    name: 'Приозерная зона',
    description: 'Зона рядом с водоёмами'
  }
]

learn> db.unicorns.find(
  { habitat: {$exists: true}},
  {_id: 0, name: 1, habitat: 1}
)
[
  { name: 'Horny', habitat: DBRef('habitats', 'mountain') },
  { name: 'Aurora', habitat: DBRef('habitats', 'forest') },
  { name: 'Nimue', habitat: DBRef('habitats', 'lake') }
]
```

Создана коллекция habitats, содержащая зоны обитания. Для нескольких единорогов добавлено поле habitat, содержащее ссылку на документ из коллекции habitats в формате \$ref и \$id.

4.2.1 Уникальный индекс по name

```
learn> db.unicorns.createIndex({name: 1}, {unique: true})
name_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_ },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.insertOne({name: 'Aurora', weight: 111})
MongoServerError: E11000 duplicate key error collection: learn.unicorns index: name_1 dup key: { name: "Aurora" }
```

Для поля name коллекции unicorns создан уникальный индекс. Повторная вставка документа с уже существующим именем Aurora вызвала ошибку дублирования ключа, значит индекс запрещает повторяющиеся значения поля name.

4.3.1 Управление индексами

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_ },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.dropIndexes()
{
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_ } ]
learn> db.unicorns.dropIndex("_id")
MongoServerError[InvalidOptions]: cannot drop _id index
```

Получена информация об индексах коллекции unicorns. Команда dropIndexes() удалила все пользовательские индексы, но системный индекс _id_ остался. Попытка удалить индекс _id_ завершилась ошибкой, так как этот индекс является обязательным.

4.4.1 План запроса и индекс numbers

```
learn> for (let start = 0; start < 100000; start += 1000) {
|   const docs = [];
|   for (let i = start; i < start + 1000; i++) {
|     docs.push({value: i});
|   }
|   db.numbers.insertMany(docs);
| }
|
| {
|   acknowledged: true,
|   insertedIds: {
|     '0': ObjectId('6a102d138850ada9efad43e3'),
|     '1': ObjectId('6a102d138850ada9efad43e4'),
|     '2': ObjectId('6a102d138850ada9efad43e5'),
|   }
| }
|
learn> db.numbers.countDocuments()
100000
```

```
learn> db.numbers.find(
|   {value: {$gte: 99996}},
|   {_id: 0, value: 1}
| ).sort({value: 1})
|
| [
|   { value: 99996 },
|   { value: 99997 },
|   { value: 99998 },
|   { value: 99999 }
| ]
```

```
learn> let noIndex = db.numbers.find({value: {$gte: 99996}}).sort({value: 1})
| ).explain("executionStats")
|
| noIndex.executionStats.executionTimeMillis
| noIndex.executionStats.totalDocsExamined
| 100000
```

```
learn> db.numbers.createIndex({value: 1})
| db.numbers.getIndexes()
|
| [
|   { v: 2, key: { _id: 1 }, name: '_id_' },
|   { v: 2, key: { value: 1 }, name: 'value_1' }
| ]
```

```
learn> let withIndex = db.numbers.find({value: {$gte: 99996}}).sort({value:
| 1}).explain("executionStats")
|
| withIndex.executionStats.executionTimeMillis
| withIndex.executionStats.totalDocsExamined
| 4
```

Создана коллекция numbers из 100000 документов. Выполнен запрос на выборку последних четырёх значений поля value. До создания индекса запрос просматривал больше документов. После создания индекса по полю value количество просмотренных документов уменьшилось, поэтому запрос с индексом является более эффективным.

Выводы:

В ходе лабораторной работы были выполнены основные операции работы с MongoDB: создание базы данных и коллекций, вставка документов, выборка данных, использование условий, сортировки, ограничения результатов, проекций и операторов для массивов. Также были выполнены запросы к вложенным объектам, работа с курсором, агрегированные запросы, изменение и удаление документов. Дополнительно были созданы ссылки между коллекциями и проверена работа индексов. На примере коллекции numbers показано, что индекс по полю value повышает эффективность поиска, так как уменьшает количество просматриваемых документов.